RESILIO

Smart Inventory. Smooth Queues.

CHAOS ENGINEERING SCRIPT

Controlled Failure Testing & Self-Healing Validation

Project	Resilio — Resilient Restaurant Management System
Technology Stack	React.js, Node.js, Express.js, MongoDB Atlas
Test Focus	Offline Inventory Recovery, Expired Stock Detection, Duplicate Token Prevention, and Queue Status Synchronization
Document Purpose	A repeatable script for demonstrating controlled failures, detection, recovery, and verification.

Testing principle: Introduce one controlled failure at a time, observe the system, and verify the final data state. A test is successful only when the expected recovery is confirmed—not merely when the UI displays a success message.

1. Purpose and Scope

This chaos engineering script defines safe, repeatable experiments for Resilio. The goal is to demonstrate how the restaurant inventory workflow behaves when connectivity is interrupted or expired stock remains in inventory. Each experiment records the injected failure, expected detection signal, healing action, and verification result.

Scope

- Offline Inventory Recovery: queue an inventory change locally while the backend is unavailable, then synchronize itafter connectivity returns.
- Expired Stock Detection: identify expired stock batches, remove remaining quantity from usable inventory, mark thebatch EXPIRED, and create an admin alert when the configured recovery endpoint supports these actions.

Out of scope

This document does not claim that every experiment is automatically healed in every environment. The actual route, database fields, logs, and recovery behavior must be checked against the current code before the demo. Use a test database or clearly labelled demo records.

2. Safety and Preconditions

1. Use a development environment and test data; do not run destructive experiments against live restaurant operations.
2. Start the React frontend and Node/Express backend, and confirm the backend health endpoint reports healthy and thedatabase is connected.
3. Keep browser developer tools and backend logs visible for evidence.
4. Record the start time, detection time, recovery start time, verification time, and final result.
5. Use a dedicated demo ingredient/batch. Back up or reset test records before repeating the experiment.
6. If an unexpected change affects real inventory, stop the test and restore the test dataset.

Readiness checklist

Check	Expected state	Pass?
Frontend	Admin dashboard and self-healing monitor load.	■
Backend	Health endpoint responds successfully.	■
Database	Test database is reachable.	■
Test data	Dedicated inventory item/batch is available.	■
Evidence	Logs, timestamps, and before/after state can be captured.	■

Additional Queue Failure Tests

- Duplicate Token Prevention: simulate two queue entries attempting to use the same token. Resilio should detect the conflict and prevent or safely resolve the duplicate so each customer has a unique token.
- Queue Status Synchronization: simulate a queue status update that fails to synchronize between the admin view, customer tracking view, and database. Resilio should detect the mismatch, restore a consistent status, and verify the saved result.

3. Experiment A — Offline Inventory Recovery

Objective: Prove that an inventory operation can be retained locally during a network interruption and synchronized when the connection is restored, without silently losing the operation or applying it twice.

Failure injection

1. Open the admin inventory screen and the self-healing monitor.
2. Start the offline recovery demo or use browser/network controls to simulate backend unavailability for the test session.
3. Add one clearly labelled test operation, for example: “Chaos Demo Tomato +1 kg”.
4. Confirm the UI indicates offline mode and that the pending operation is stored in the browser’s local IndexedDB queue.
5. Restore the network/backend connection and observe the synchronization process.

Expected detection and healing

- Detection: the application recognizes that the request cannot reach the backend or that connectivity is unavailable.
- Containment: the operation is placed in the local pending queue rather than being discarded.
- Recovery: after connectivity returns, the queued operation is submitted to the configured offline-sync endpoint.
- Verification: the server acknowledges the operation, the pending queue is cleared only after success, and the finalinventory quantity matches the expected result.

Pass criteria

Pass only if the operation remains available while offline, synchronizes after reconnection, produces the expected final quantity exactly once, and leaves no failed item silently marked as complete. If duplicate application or data loss occurs, record the test as failed and preserve the logs.

Evidence to capture

Screenshot/video of offline status, pending queue count, connection-restored event, sync result, and before/after inventory quantity. Record all timestamps and calculate recovery time as verification timestamp minus failure-injection timestamp.

4. Experiment B — Expired Stock Detection

Objective: Validate that expired stock is detected and made unavailable for use, and that an administrator receives a useful alert.

Failure injection / setup

1. In the test database, select or create a dedicated stock batch with an expiry date earlier than the test date and aremaining quantity greater than zero.
2. Ensure the batch is not already marked EXPIRED or DEPLETED.
3. Open the inventory/self-healing screen and trigger the expired-stock check or the configured expiry recovery action.
4. Watch the timeline and backend logs; do not assume a button click alone proves that detection occurred.

Expected detection and healing

- Detection: the expiry check finds a batch whose expiry date is before today and whose remaining quantity is greaterthan zero.
- Healing: the expired quantity is recorded, remaining usable quantity is set to zero, and batch status becomes EXPIRED.
- Notification: an admin alert identifies the batch, ingredient, expired quantity, expiry date, and corrective action.
- Verification: query or refresh the persisted batch record and confirm it cannot be treated as usable stock.

Pass criteria

Pass only if the database state is updated correctly, the expired quantity is accounted for, the batch is excluded from usable stock, and the admin alert is created. If the UI reports “recovered” but persisted data is unchanged, mark the experiment failed.

Evidence to capture

Before/after batch state, expiry date, remaining quantity, expired quantity, admin alert, backend log, and recovery timeline. Use a test batch only; never alter a real batch for demonstration.

5. Experiment C · Duplicate Token Prevention

Objective

Verify that two queue entries cannot end up with the same token and that a duplicate-token conflict is detected and handled safely.

Failure injection

1. Use test queue records in the development environment.
2. Trigger or simulate two queue requests that attempt to claim the same token.
3. Watch the Self-Healing Monitor and backend logs for the duplicate-token detection signal.

Expected detection and healing

Detection: the system identifies that a token is already assigned or that a token collision has occurred.
Healing: reject the conflicting assignment or generate/assign a unique token, depending on the implemented recovery logic.
Verification: confirm every active queue entry has a unique token in the persisted database and that the customer/admin views show the same token.

Pass criteria

Pass only if the duplicate is detected, the final queue data contains no conflicting active tokens, and the UI and database agree. If the UI says recovered but duplicate tokens remain, record the experiment as failed.

Evidence to capture

Before/after queue records, duplicate-token alert or log, recovery timeline, and confirmation that the final tokens are unique. Use demo records only.

6. Experiment D · Queue Status Synchronization

Objective

Verify that queue status stays consistent across the admin Queue Management page, customer token tracking, and the saved database record after a status update.

Failure injection

1. Create a clearly labelled test queue entry.
2. Simulate or trigger a status update that does not synchronize correctly, such as an interrupted update or a stale status inone view.
3. Observe the Self-Healing Monitor, API/backend logs, and both admin and customer views.

Expected detection and healing

Detection: the system identifies a mismatch between the saved queue status and the status displayed by a client view. Healing: re-fetch or reconcile the authoritative status and update the stale view/record using the recovery behavior implemented in Resilio.
Verification: refresh both views and confirm the database and UI agree on the same valid status: Waiting, Your Turn, Seated, or Completed.

Pass criteria

Pass only if the mismatch is detected, the final persisted status is correct, and the admin and customer views show the same status. If the status differs after refresh or the database is unchanged, mark the test as failed.

Evidence to capture

Before/after status values, incident/log entry, recovery timeline, and screenshots of the admin view, customer tracking view, and persisted record.

5. Recording Results

Complete this table during the live demonstration. Use measured values from logs and persisted data; do not enter estimated recovery times as if they were measured.

Experiment	**Failure injected**	**Detection signal**	**Healing action**	**Measured recovery time**	**Result**
Offline Inventory Recovery	Temporary loss of backend/network access	Offline state / failed request / queued operation	Sync queued operation after reconnect	_____ sec	Pass / Fail
Expired Stock Detection	Expired batch with remaining quantity	Expiry check identifies eligible batch	Zero usable quantity, mark EXPIRED, alert admin	_____ sec	Pass / Fail

Suggested event log

Timestamp	**Event**	**Observed evidence**
__________	Failure injected	What was deliberately interrupted or made invalid?
__________	Failure detected	Which signal/log/state confirmed detection?
__________	Healing started	What recovery action began?
__________	Verification complete	Which persisted state proved recovery?

6. Rollback and Cleanup

1. Stop the failure simulation and restore normal network/backend settings.
2. Confirm the backend health endpoint and database connection are healthy.
3. Remove or reset demo-only inventory records and alerts if required by the project workflow.
4. Confirm no pending offline operations remain, or document any unresolved item before clearing the test data.
5. Save screenshots/logs and complete the results table.

7. Limitations and Honest Reporting

Chaos testing is evidence-based. Report only behavior observed in the current implementation. If detection did not trigger, the UI displayed an error, the request timed out, or recovery was incomplete, record that outcome as a failure or limitation. Do not label a test “self-healed” until the database state and user-visible result have both been verified.

End of Script • RESILIO